Formal Methods in Software Engineering

Christian Rinderknecht
Christian.Rinderknecht@tezcore.com
Nomadic Labs (Paris)
19 October 2018

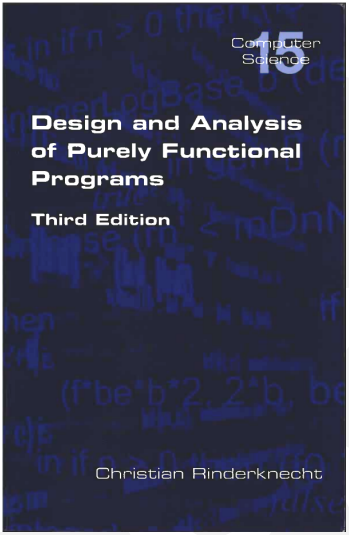
Tezos Masterclass

Organisation and contributions

Caleb Kow
Diego Olivier Fernández Pons
Tezos SouthEast Asia
Singapore

- My alma mater is *Université Pierre et Marie Curie* (UPMC, a.k.a. Paris 6).
- I did my doctoral studies at **Inria**, one of the most prestigious research institutes in informatics in France.
- I was a member of the team that developed the programming language OCaml.
- I went on to work as an engineer, a researcher and a professor for many years, across several countries (France, Korea, Hungary, Sweden), both in academia and private industries.
- I recently joined the core development team behind the **Tezos** blockchain, where I will be caring for compiler construction and domain-specific languages.

I have written a book about functional programming.



Introduction

- This lecture is meant to present what **formal methods** are, in particular in the context of **software engineering**.
- It provides you with the industrial context and theoretical framework within which you can proceed to practical knowledge.
- Some of these formal methods and tools are used by **Tezos**, or may be used.
- For instance, I will teach you a bit of OCaml, which is used for all the code base of the **Tezos** blockchain.
- This is the first of a series of lectures which hope to entice you to think how to contribute academic research or to become a successful engineer in leading or emerging industries (transport, aerospace, smart cards and... **blockchain**!).

Complex software is everywhere... and has bugs

Ariane 5. Credit: ignis, GFDL, CC BY-SA-2.5,2.0,1.0.

The case of Ariane 5

- In June 1996, the maiden launch of the European rocket Ariane 5 ended in its self-destruction.
- The cause was in the software controlling the trajectory.
- This piece of software was designed with the requirements of Ariane 4, **not** Ariane 5.
- Accordingly, some ranges of physical parameters were taken from Ariane 4, and wrongly assumed in the programming of Ariane 5.
- Therefore, certain errors were deemed impossible for physical reasons and not handled in the code.
- During the flight, a value overflowed, causing an exception to be raised, but not handled, which lead to the self-destruction of the launcher (the control system believed that the rocket was off-course).

The case of Ariane 5

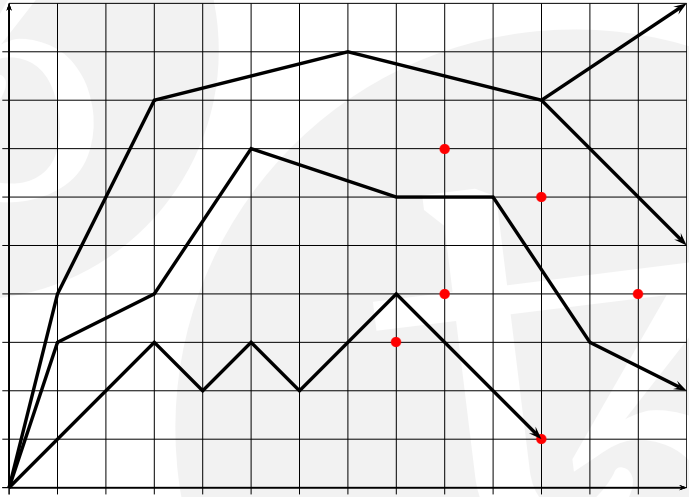
- The language used to program that piece of software in Ariane 5 was Ada.
- Ada is a programming language originally designed in France in the early 80s, before it became the official language for most developments for the defence of the USA.
- Ada is used a lot in critical embedded systems, deployed in transport, aerospace and defence.
- The language was carefully designed so it lends itself well to all sorts of **static analyses**.
- These analyses are static because they are performed by the Ada compiler itself, or by other tools that read the source code.
- Their aim can vary, but perhaps the most important is **the detection of run-time errors** before the code is deployed.

- The catastrophic failure of the first Ariane 5 incurred a very high cost, and the loss of ten years of research and development in satellite technology.
- Ada itself is not to blame.
- Instead of going into the details of what went wrong, let us consider what happened next.
- Some researchers at **Inria** (Gonthier & Deutsch) proposed a static analyser for Ada that could have caught the error that lead to the loss of Ariane 5, and discovered many other errors.
- The mathematical theory they used to analyse the software is called **abstract interpretation**.
- This powerful theory was invented in the 70s by the French researchers Patrick and Radhia Cousot.

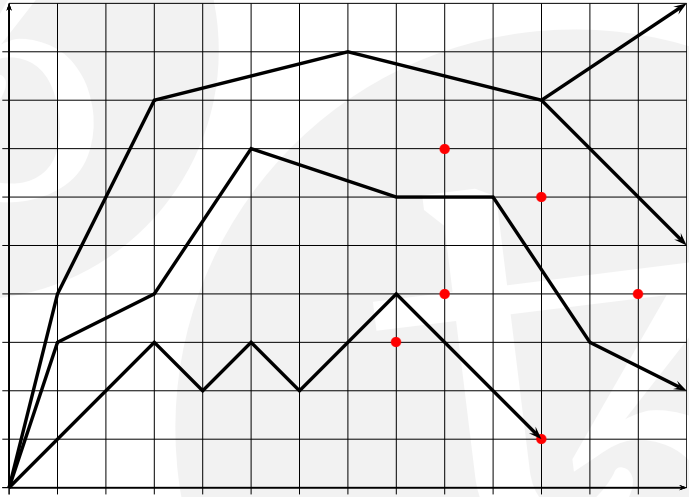
- In 1999, a start-up company called **PolySpace Technologies** was founded in France by Alain Deutsch (**Inria**) and Daniel Pilaud (co-inventor of Lustre, a declarative language for programming synchronous systems).
- I joined that company in 2000 to work on the static analysis of JavaCard.
- The idea was to translate JavaCard and other languages (C and C++) to Ada, for which there was already a static analyser, following the explosion of Ariane 5.
- The translation did not need to preserve all behaviours, only those who could go wrong (run-time errors).
- The company was a success and is now part of **Mathworks Inc.**, known for MATLAB and Simulink.
- PolySpace (the tool) is well known in the avionics industry.

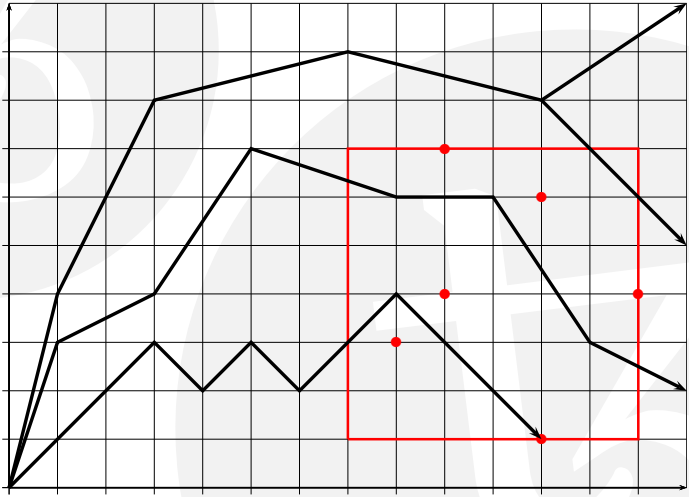
- **Abstract interpretation** is a general framework to analyse programs and systems.
- It makes an **over-approximation** of the behaviours of a program, then deduce properties on those, and finally translates back the abstract properties in terms of the original behaviours of the program.
- Abstract interpretation is used to replace a system with a possibly infinite or very large number of states with a system that has a small number of states that can all be checked exhaustively.

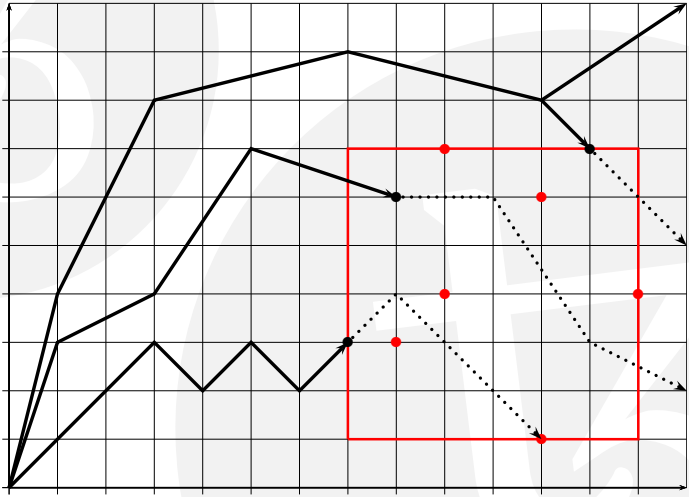
- Abstract interpretation can also be used to **prove the absence of errors**, or to find errors.
- As an illustration, let us consider a system whose states are pairs of positive natural numbers, represented as coordinates on a quarter of plane.
- A **state** is all the information characterising a system running at a given point in time (memory contents, program counter, files, network buffers, etc.)
- Some states are coloured in red to denote that they are erroneous: this is the property "To be erroneous."
- A behaviour of a system, or **trace**, can be thought as a line from one state to the next. We show four traces as black, thick lines.
- One of the traces actually ends in an error.



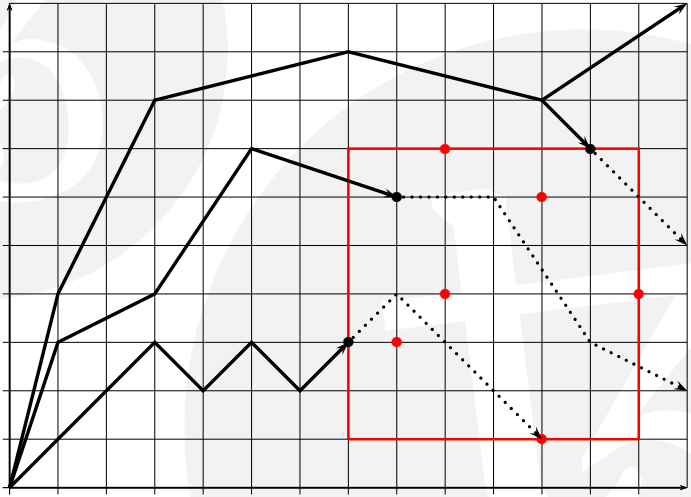
- We need to characterise the erroneous states in a simple manner, at the cost of over-approximating them: this means to abstract the property "To be erroneous" so it becomes easier to check.
- For example, we can abstract the error property by a red square containing the set of erroneous states, instead of being the set itself.
- A square can be defined by three numbers only, and it is easy to check whether a given state belongs to it: if so, we deem it an error.
- In our example, this creates **false positives**, that is, states inside the square that are not actual errors.

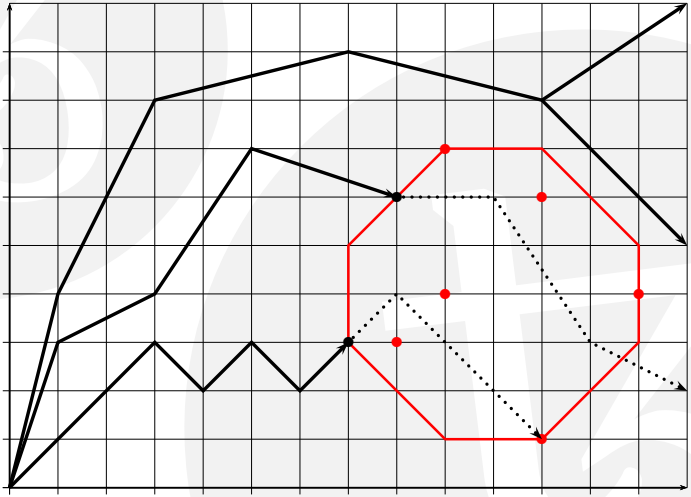






- To reduce the number of false positives, we **refine the abstract property** so it fits better the erroneous states.
- For instance, we could use another polygon to cover more tightly the erroneous states.
- Let us try an hexagon: to define it, we need the same amount of information as for a square, but the inclusion property is slightly more complex.
- This is better, but one trace remains a false positive.
- In general, false positives are inevitable if the property is complex enough, so trade-offs are necessary.





A success story: the Parisian métro line 14

Métro line 14. Credit: Pline, CC BY-SA-3.0

The Parisian métro line 14

- In October 1999, opened in Paris the first entirely automated metro line (number 14).
- Some people thought that the trains were controlled at a distance, but they actually moved and stopped based on decisions taken by themselves, with inputs from sensors and communication networks.
- To ensure such a high-level of safety and continuity of service, the company Matra Transport used formal methods from the start of the project.
- After the success of that new metro line, an existing metro line (number 1) has been automatised successfully.
- **Railways signalling systems** are a major user of formal methods, due to the high safety requirements.

The Parisian métro line 14

- The train system comprises four subsystems:
  1. the automatic pilot and signalling system,
  2. the operation control centre,
  3. the platform doors,
  4. the audio-video system.
- Only the first subsystem includes software that is critical to the safety of the passengers.
- It controls the movement of trains (speed, traction power, routes) and the doors (on-board and on the platforms).
- It sends any alarm to the operation control centre.

The Parisian métro line 14

- The system architecture is naturally distributed and is made of
  1. the line equipment, in the operation control centre,
  2. the wayside equipment along the tracks,
  3. the on-board equipment.
- They are interlinked via a high-speed communication network.

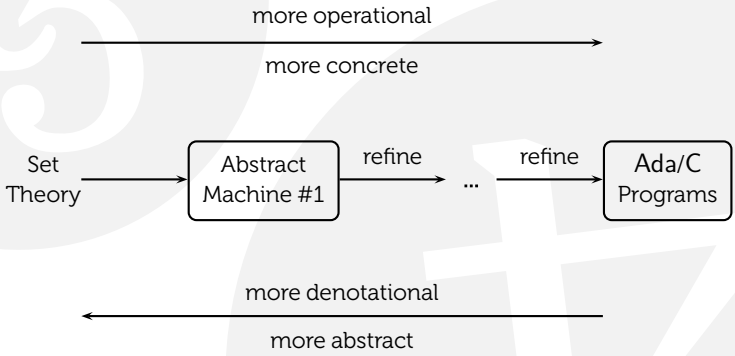
- In the operation control centre, operators monitor scheduled train programs for each day and supervise events received from the network.
- Platform doors prevent people from falling on the tracks. The train doors open only when they are aligned with the platform doors.
- If not properly aligned, there are emergency doors on the platform that can be manually opened.
- Audio and video feeds allows operators to monitor the inside the carriages and to speak with passengers (a high-level of availability is required to ensure security).

The Parisian métro line 14

- In order to bring strong correctness assurances in software, it is paramount to carefully specify the expected behaviours and delimit the field of application of the mathematical proofs.
- The use of the **B method** by Matra Transport International (now part of Siemens) for the Parisian metro line 14 is a case in point of this approach.
- Another example is the Calcutta metro, in 1992.
- The B method was invented in the 80s by Jean-Raymond Abrial, a French researcher.
- It comprises
  1. a formal notation to expression specifications,
  2. a design methodology,
  3. and software tools as a support.

- The formal **B language** is based on **set theory**, which makes it very expressive.
- It enables the specification of a system and its expected properties into a model called an **abstract machine**.
- It also enables the proof of such properties (like safety): some as **system invariants** (properties that must hold on all states), some as **postconditions** to mathematical functions.
- A **proof assistant** then either automatically proves some of these theorems, or requests the engineer to prove them interactively.
- The B method and its software tools allow the engineer to **refine** the abstract machine into a series of abstract machines until a **concrete machine** is obtained.
- A concrete machine is one that can be translated automatically into a programming language (C or Ada).

- Each abstract machine is, in turn, less and less **denotational**, and more and more **operational**.
- "Operational" means "what a system does", whereas "denotational" means "what properties the system satisfies".
- Step by step, the abstract machines become closer to **algorithms**, that is, functions operating over data structures, which is what a concrete machine is about.
- The other aspect of a concrete machine is that it is **deterministic**: there are no unspecified choices left in the control flow ("what to do next" is always explicit).


more operational
more concrete
Set Theory
Abstract Machine #1
refine
...
refine
Ada/C Programs
more denotational
more abstract

- An abstract machine in the B language consists of
  - **states**, that is, variables satisfying some invariant properties, and
  - **operations**, that is, functions that change the state while maintaining the invariants, and return values.
- The variables making up the state of an abstract machine can denote mathematical sets, relations, functions, etc.
- Operations are defined with
  - **preconditions**: "What is assumed about the state by the operation before starting", and
  - **postconditions**: "What can be assumed about the state and return value after the operation is finished".

- Let us specify in the B language what is the Greatest Common Divisor (GCD) of two natural numbers.
- In English: "The GCD of x and y is d such that d divides both x and y, and any divisor of x and y is lower than or equal to d."
- In mathematical notation:
$$\gcd(x,y) \mid x \quad \wedge \quad \gcd(x,y) \mid y \quad \wedge \quad \forall z.(z \mid x \wedge z \mid y \Rightarrow z \leqslant \gcd(x,y)).$$
- The next slide shows the equivalent definition in the B language (note that variables must be at least two characters long).

Greatest Common Divisor (abstract)

```
MACHINE gcd /* Greatest Common Divisor */
OPERATIONS
  rr <-- gcd (xx,yy) =
    BEGIN
      PRE xx : INT & 1 <= xx & xx < MAXINT
          yy : INT & 1 <= yy & yy < MAXINT
      THEN
        ANY dd WHERE
         dd : INT
           & (xx-(xx/dd)*dd = 0) & (yy-(yy/dd)*dd = 0)
           & !zz.(zz : INT & zz < MAXINT
                 & (xx-(xx/zz)*zz = 0)
                 & (yy-(yy/zz)*zz = 0) => zz <= dd)
        THEN rr := dd END
      END
    END
```

- The refinement from the initial abstract machine to the final concrete machine is stepwise and the engineer must provide more details about the low-level aspects of the system, whilst proving the new specification is **correct and complete** with respect to the previous one (all its behaviours are expected by the previous one, and vice-versa).
- This transitively entails that the concrete machine is **equivalent** to the initial abstract machine.
- The last step consists in the **generation of Ada programs** which are thus, by construction, correct and complete with respect to the initial specification.

Greatest Common Divisor (concrete)

```
REFINEMENT gcd_R1 /* refinement of ... */
REFINES gcd /* the previous machine */
OPERATIONS
 rr <-- gcd (xx,yy) = /* No room to handle xx < yy */
   BEGIN
     PRE xx : INT & 1 <= xx & xx < MAXINT
         yy : INT & 1 <= yy & yy < MAXINT
     VAR rem, old_xx, old_yy IN
     WHILE (yy != 0)
       old_xx := x; old_yy := yy; rem := xx-(xx/yy)*yy;
       xx := old_yy; yy := rem;
     INVARIANT gcd (old_xx, old_yy) = gcd (old_yy, rem);
     VARIANT yy < old_yy; /* Termination */
     rr := xx;
   END
```

From the concrete machine, the following Ada code fragment would be produced, were the tooling told to inline the values of the variables `rr`, `old_xx` and `old_yy`, which are only needed for logical reasons (to state the invariant and variant, and the final result), not computational ones.

```
function GCD (Xx, Yy : Integer) return Integer is
  Rem : Integer;
  while Yy /= 0 loop
    Rem := Xx mod Yy;
    Xx := Yy;
    Yy := Rem;
  end loop;
  return Xx;
end GCD
```

Introduction (resumed)

- Formal methods are methods and tools for software development that are grounded in theoretical computer science (informatics).
- Some expected benefits are:
  - the reduction in maintenance costs (e.g., corrective maintenance),
  - more robustness (by explicitly delineating the domain of validity) and, more generally,
  - higher reliability and levels of confidence.
- Formal methods are often applied in the making of **critical embedded systems** to guarantee safety and security, whenever human lives or large investments depend upon their correctness and continuity of service.

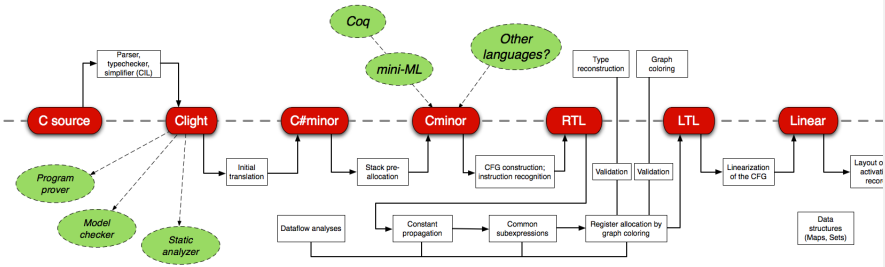
Airbus A340. Credit: Konstantin von Wedelstardt. GFDL 1.2

- The usefulness of formal methods should not be underestimated. They have taken decades to mature from academic research to practical frameworks, and are becoming more and more widespread.
- Beyond PolySpace and the B method, we can mention some other successful static analysers.
- **Astrée** offers an abstract interpretation of C programs to detect run-time errors, in the same vein as PolySpace.
- It was designed at École Normale Supérieure (France) and is now commercialised by the German company AbsInt.
- In 2003, Astrée proved automatically (on a standard PC) the absence of any run-time errors in the primary flight control software of the Airbus A340 fly-by-wire system: 132,000 lines of C.
- Astrée is written in OCaml and PolySpace in Standard ML, the American cousin of OCaml.

Dassault Falcon 900. Credit: Aero Icarus, CC BY-SA-2.0

- Another well-known static analyser for C is **Frama-C**.
- It was designed by researchers at the CEA (France).
- Frama-C is actually a software suite which includes a static analyser, but also software to interactively prove that the source code satisfies some **functional specifications** provided by the user.
- Recently, Frama-C has been used by Dassault Aviation to detect common weaknesses in real-time software running on the Falcon jets.
- In 2016, I joined the French company **TrustInSoft** that commercialises bespoke plug-ins for Frama-C.
- The code base of Frama-C is almost entirely written in OCaml (perhaps the largest OCaml code base in the world).

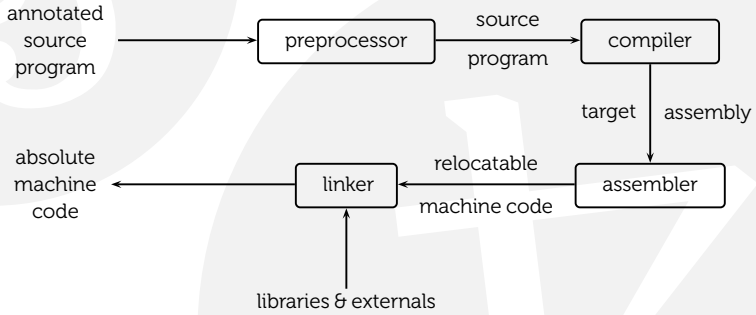
Formal methods applied to compilers



Formal methods applied to compilers

- Formal methods can also be applied to **compilers**.
- Compilers translate programs of high-level languages ("close to human thought"), like C, Ada or OCaml into programs of low-level languages ("close to machine thought"), like assembly or byte-code, and these two programs must be **operationally equivalent**.
- This means that all behaviours of the input program must be preserved in the output program (**completeness**) and all behaviours of the output must map to behaviours of the input (**soundness**, also called *correctness*).
- Since C is widespread in critical embedded software, it is desirable to consider C compilers.

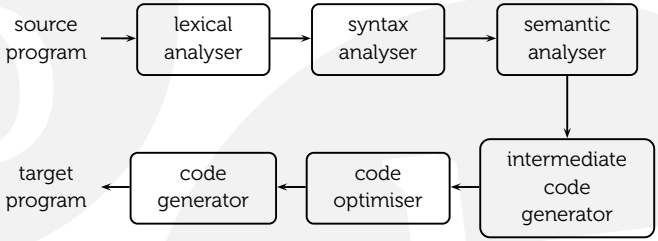
General architecture of a C compiler



There are two phases to compilation: **analysis** and **synthesis**.

1. The analysis breaks up the source program into constituent pieces of an intermediary representation of the program.
2. The synthesis constructs the target program from this intermediary representation.
   - Actually, there can be many intermediary representations, each one getting closer to the target language.
   - Each phase is itself decomposable into other phases, depending on the source and target languages.

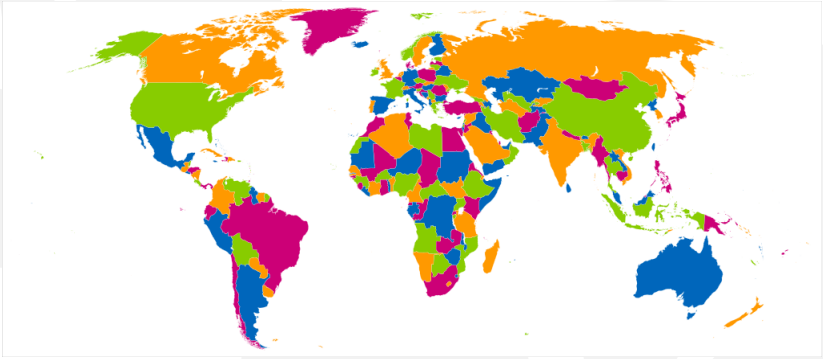
The first row makes up the analysis and the second is the synthesis.



- A real compiler has many more stages than shown above, and is always a complex piece of software where more errors are likely.
- Those errors can be silent: some erroneous code can be emitted only for certain classes of programs, which is a serious problem for embedded applications.
- Xavier Leroy at **Inria** (France) has spearheaded the design and implementation of a **certified C compiler**, called **CompCert**, using Coq.

- CompCert is certified because its **OCaml code has been extracted from the specification** (of a large subset) of C and intermediary representations, as well as from **semantic-preserving transformations** between those representations, down to optimised assembly.
- John Regher and his colleagues are experts in finding bugs in C compilers. They wrote, in 2011:

> *The striking thing about our CompCert results is that the middle-end bugs we found in all other compilers are absent. [...] This is not for lack of trying: we have devoted about six CPU-years to the task. The apparent unbreakability of CompCert supports a strong argument that developing compiler optimizations within a proof framework, where safety checks are explicit and machine-checked, has tangible benefits for compiler users.*



Political map of the world. Credit: Fibonacci, CC BY-SA 3.0

- Coq is a **proof assistant**, that is, a piece of software in which to express logical formulas, prove them and have the proofs checked automatically. It is an interactive tool, but it automates the search of some proof patterns (libraries called **tactics**).
- Coq comes with a programming language named **Gallina** which is close to OCaml, but **without side-effects** and adding **dependent types**. This type system is more general than the type systems found in usual languages like Java.
- Coq features **code extraction**: it can generate an OCaml program from the proof of its properties.
- Code extraction relies upon the **Curry-Howard correspondence**, which states that a proof is analogous to a function, and the proved theorem is analogous to the type of that function.
- Coq was created in 1989 by Huet and Coquand, at **Inria** (France), where it is developed today. It is written in OCaml.

Here is a list of compilers certified by Coq:

- CompCert, an optimizing compiler for a subset of C by Xavier Leroy in 2012;
- CertiCoq, a compiler by researchers at Princeton, Edinburgh, Inria and Cornell in 2017;
- Q*cert, a query compiler by IBM Research in 2017;
- Ergo, a compiler from Ergo (a language of legal smart contracts) to Michelson by Jerôme Siméon in 2018.

In graph theory, it was used to prove the **four colour theorem**, that states that countries in an arbitrary map can always be coloured with four colours (Gonthier, Microsoft Research, 2008).

Heartbleed bug logo. Credit: Codenomicon, CC0 1.0

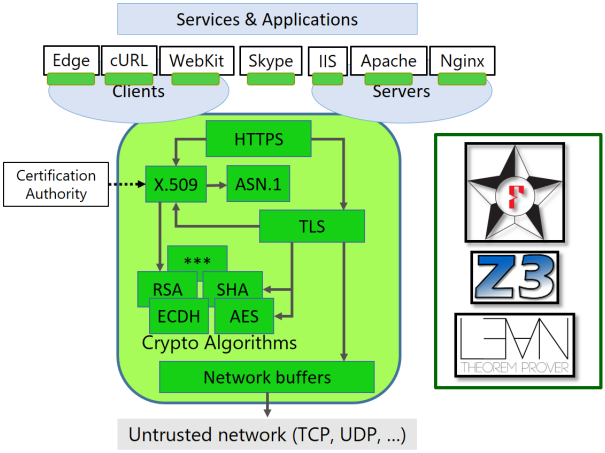
- In April 2014 was disclosed a bug in the OpenSSL cryptographic library introduced in 2012. The bug was named **Heartbleed** and given a logo to increase public awareness.
- The bug is due to improper input validation (missing bounds check) in the implementation of the TLS heartbeat extension and allows protected data to be read. The bug allows stealing private keys and users' session cookies and passwords
- The faulty code was written by a PhD student at Fachhochschule Münster, and the OpenSSL core developer that reviewed the code did not notice the problem.
- Around 500,000 SSL certificates were compromised by the bug and had to be reissued.
- Canada Revenue Agency was stolen 900 social insurance numbers in 2014 due to the bug. The records of 4.5 million patients were accessed after a breach into the system of Community Health, a private hospital in the USA.

Given how critical cryptographic primitives are, **Tezos** uses HACL*, a formally verified cryptographic library, developed by the Prosecco team at Inria in collaboration with Microsoft Research.

- HACL* is written in F*, an ML-style general-purpose functional programming language (with effects) aimed at program verification. F* is a cousin of OCaml and Coq.
- The F* code is then extracted into C with the KreMLin tool.
- To generate C code that is proven is correct, the F* code is first transformed into Low*, a shallow embedding of a subset of C into F* with explicit memory management. The code is compiled from Low* to C and compiled with a C compiler (certified like CompCert, or not)

According to the authors of HACL*,

> *when compiled with GCC on 64-bit platforms, our primitives are as fast as the fastest pure C implementations in OpenSSL and Libsodium, significantly faster than the reference C code in TweetNaCl, and between 1.1× and 5.7× slower than the fastest hand-optimized vectorized assembly code [...]*


Services & Applications
Edge
cURL
WebKit
Skype
IIS
Apache
Nginx
Clients
Servers
HTTPS
Certification Authority
X.509
ASN.1
TLS
***
RSA
SHA
ECDH
AES
Crypto Algorithms
Network buffers
Untrusted network (TCP, UDP, ...)
Z3
THEOREM PROVER

F*

F* is an ML-style general-purpose functional programming language with effects aimed at program verification.

- It was designed and developed at the joint Microsoft Research-Inria Center.
- F* programs can be extracted to efficient OCaml, F♯, or C code.
- The F* typechecker aims at proving that programs meet their specifications using a combination of **SMT solving** and interactive proofs.
- Like Coq, F* can also be helped with intermediate lemmas and tactics when it cannot prove automatically the properties of the code.

There is an online tutorial `https://www.fstar-lang.org/tutorial/`

Here is the factorial function in F*:

```
val fact: n:int{n ⩾ 0} → Tot int

let rec fact n =
  match n with
    0 → 1
  | _ → n * fact (n-1)
```

The annotation `n:int{n ⩾ 0} → Tot int` is requesting the F* compiler to prove that the factorial function terminates (`Tot`) and returns a positive integer (`{n ⩾ 0}`) when given as input a positive integer.

- F* proves the function terminates using **well-founded recursion**.
- If the positivity constraint in the input `n:int{n ⩾ 0}` is removed, the type-checker of F* informs that it cannot prove the property.
- Indeed, if $n < 0$, the function does not terminate and a stack overflow occurs.

CPU Intel Pentium, 66 MHz. Credit: Konstantin Lanzet, CC BY-SA 3.0

The Intel Pentium bug

- The floating-point unit (FPU) of some Pentium processors (Intel) contained an error when a certain kind of division was performed.
- The FPU is a numeric coprocessor, to which the main processor (CPU) delegates computations for a speed up (about $10\times$).
- The error was discovered in 1994 by Thomas Nicely, a professor of mathematics at Lynchburg College (USA), who was performing number theoretic calculations.
- The division *"1/824633702441.0 is calculated incorrectly (all digits beyond the eighth significant digit are in error)." – Thomas Nicely.*
- Intel had to recall all faulty units, incurring a large cost, but also a very bad press (CNN) in what was at the time a competitive market.

- That bug is known as the "Intel Pentium FDIV bug", after the instruction for floating-point division.
- Intel reacted by investing in formal methods for the design of its chips.
- For example, John Harrison (Intel) has formally verified the correctness (of rounding) of various floating-point algorithms using the **HOL Light prover** (a cousin of Coq, also written in OCaml).
- Also, various mathematical theorems in elementary number theory and real analysis were proved within the same framework.
- Intel combined theorem proving with a more automated approach when the state space is finite and small enough: **model checking**.

- **Model checking** was invented in the 80s to check the properties of concurrent systems. (Clarke, Emerson, and Sifakis shared the 2007 Turing Award for their work.)
- Model checking reasons on the **states of a program**.
- It requires a way to define
  - all possible states of a program, including those that should not be reached, and
  - the execution of a program as the **transition** from one state to another (**traces**).
- There is usually a very large, but **finite number of states**.

- **Temporal logic** is used to express properties of states
  - **Safety**: a given ("bad") state can never be reached;
  - **Liveness**: a given ("good") state will eventually be reached;
  - **Reachability**: a given ("good") state can be reached.
- States and sets of states are usually defined with **logical formulas**.
- **SMT solvers** are often used to find a trace that leads to an undesirable state or prove the state is unreachable by exhausting all traces.

The takeaway

What have you learnt in this lecture?

The takeaway

- Bugs are everywhere and can be very costly.
- The **B method** helps manually narrowing the gap between specification and concrete implementation.
- **Abstract interpretation** runs an approximation of a program on an approximation of all possible inputs.
- Astrée, PolySpace and Frama-C are abstract interpreters written in OCaml, used in the aerospace industry.
- Coq is a **proof assistant** used to check mathematical theorems.
- Coq is also used to write **certified compilers**.

The takeaway

- F* enables proving properties of functions using **type annotations**.
- F* automatically proves the termination of recursive functions using **well-founded recursion**, but does not always succeed.
- F* has been used to write efficient certified cryptographic libraries.
- **Model checking** is a technique to analyse all possible states of a program and prove that some (bad) states are unreacheable.
- States and sets of states are defined with **logical formulas**.
- **SMT solvers** are used to find a trace reaching undesired states, or prove the absence of such a trace.